Vélo'v : aide-mémoire des commandes

29 octobre 2026 · Estéban

Tu codes sur ton Mac, tu pousses vers la VM 206 (192.168.1.232), et la VM fait tourner les services. Chaque section dit où se lance la commande. Rédigé avec l'aide d'une IA (Claude).

Où se lance quoi

Action	Où	Pourquoi
Écrire le code, commiter	Mac	Un seul endroit où l'on modifie le code, sinon conflits avec Fabien
Tests, entraînements de mise au point	Mac	Rapide, sans risque
Serveur MLflow	VM	Toujours allumé, partagé par le binôme
Collecte des données réelles (cron)	VM	Doit tourner jour et nuit, même Mac éteint
API en production	VM	Joignable depuis un autre poste

Repère : ton invite affiche `estebancosta@MacBook-Air...` sur le Mac et `groupe6@groupe6` sur la VM. Sur le Mac, elle doit commencer par `(.venv)` avant toute commande Python du projet.

Se connecter à la VM et y envoyer son code

Prérequis (déjà fait) : être sur le Wi-Fi de l'environnement de l'école, la clé `~/.ssh/groupe6` en droits 600, et ce bloc dans `~/.ssh/config` sur le Mac :

```
Host 192.168.1.232
    User groupe6
    IdentityFile ~/.ssh/groupe6
```

Je veux…	Commande (depuis le Mac)
Ouvrir un terminal sur la VM	`ssh 192.168.1.232` (revenir : `exit`)
Lancer une seule commande sur la VM	`ssh 192.168.1.232 "commande"`
Envoyer ma branche à la VM	`git push vm tp4`
La sauvegarder sur GitHub	`git push`
Récupérer le code sur la VM	`ssh 192.168.1.232 "cd velov-mlops && git pull origin tp4"`
Voir les dépôts distants	`git remote -v`

Les trois dépôts distants du Mac :

Nom	Adresse	Usage
`origin`	GitHub, ton fork `Choobzed/velov-mlops-chanpiu-costa`	Sauvegarde et rendu (tags)
`cours`	GitHub, `MohamedGassem/velov-mlops-student`	Lecture seule : récupérer les fichiers du prof
`vm`	`groupe6@192.168.1.232:velov.git`	Déployer sur la VM

Sur la VM, `~/velov.git` reçoit les push (dépôt nu, sans fichiers visibles) et `~/velov-mlops` est la copie de travail où l'on fait `git pull origin tp4` et où tournent les stacks Docker. Ne jamais modifier le code directement dans `~/velov-mlops` : tout passe par le Mac, puis `git push vm`.

Récupérer un fichier fourni par le prof (sans fusionner le reste) : `git fetch cours` puis `git checkout cours/main -- chemin/du/fichier`.

Sur la VM, `git remote -v` n'affiche qu'un `origin`, qui désigne le dépôt nu `~/velov.git` : c'est normal, la VM ne fait que recevoir. **`origin` ne désigne donc pas la même chose sur le Mac (GitHub) et sur la VM (dépôt nu).**

Sécurité facultative, pour qu'un `git push cours` par erreur échoue tout de suite : `git remote set-url --push cours PAS_DE_PUSH_VERS_LE_COURS` (la lecture avec `git fetch cours` continue de marcher).

Docker : sur le Mac ou sur la VM (le contexte)

La commande `docker` est une télécommande : elle envoie des ordres à un moteur Docker. Le **contexte** choisit quel moteur reçoit l'ordre. Le code Python, lui, s'exécute toujours sur la machine où tu tapes la commande : le contexte ne concerne que Docker.

```
terminal du Mac ── docker ...                ──────────► moteur Docker du Mac (Docker Desktop)
terminal du Mac ── docker --context vm ... ── SSH ──► moteur Docker de la VM
```

Avec `--context vm`, Docker lit sur le Mac le fichier compose, le `.env` et les fichiers à construire, puis les envoie à la VM : l'image est construite et les conteneurs tournent **sur la VM**, avec ses volumes. Le Mac peut ensuite s'éteindre.

Je veux…	Commande (depuis le Mac)
Créer le contexte (déjà fait, une seule fois)	`docker context create vm --docker "host=ssh://groupe6@192.168.1.232"`
Lister les contextes (`*` = celui par défaut)	`docker context ls`
Voir les conteneurs de la VM	`docker --context vm ps`
Construire et lancer MLflow sur la VM avec mes fichiers locaux	`MLFLOW_PORT=5000 docker --context vm compose -f compose.mlflow.yaml up -d --build`
Voir l'état de MLflow sur la VM	`docker --context vm compose -f compose.mlflow.yaml ps`
Lire les logs de MLflow sur la VM	`docker --context vm compose -f compose.mlflow.yaml logs mlflow`

Trois règles :

- Toujours `--context vm` **commande par commande**, jamais `docker context use vm` : sinon toutes les commandes suivantes visent la VM sans que tu le voies.
- Le `.env` lu est celui **du Mac** : il doit contenir le **même** `MLFLOW_DB_PASSWORD` que la VM (la base garde le mot de passe de sa création). Pour le recopier sans le retaper : `ssh 192.168.1.232 "grep MLFLOW_DB_PASSWORD velov-mlops/.env" >> .env`.
- `MLFLOW_PORT=5000` devant la commande évite que le `MLFLOW_PORT=5001` du Mac (AirPlay) s'applique à la VM.

	`docker --context vm compose ... --build`	**`git push vm tp4` puis `ssh ... git pull ... up -d --build`**
Fichiers utilisés	Ceux du Mac, même non commités	Ceux d'un commit précis
`.env` lu	Celui du Mac	Celui de la VM
Usage	Tester vite une modification	Déployer une version propre, retrouvable par Fabien ; rendu

Limite : la stack de l'API (`compose.yaml`) lit ses mots de passe dans des fichiers `secrets/`, que Docker chercherait alors sur le disque de la VM. Pour elle, passer par `git push vm` puis `ssh`, après avoir créé ces fichiers sur la VM.

Comment le Mac et la VM communiquent

Voie	Sert à	Exemples
SSH (clé `groupe6`)	Donner des ordres, transférer du code	`ssh 192.168.1.232`, `git push vm tp4`, `docker --context vm ...`
HTTP, port 5000	Parler au serveur MLflow	Navigateur sur `http://192.168.1.232:5000`, `python -m velov.train --mlflow`

Cas typique du TP4 : l'entraînement **calcule sur le Mac** et **stocke sur la VM**, en envoyant ses runs par HTTP au serveur MLflow.

Préparer l'environnement Python (Mac)

À chaque nouveau terminal, depuis la racine du projet :

```
cd ~/Desktop/cours/ynov20262027/iacloud/velov-mlops-student
conda deactivate
source .venv/bin/activate
which python
```

`which python` doit finir par `velov-mlops-student/.venv/bin/python`. Si l'invite affiche `(base)`, c'est Anaconda : refaire `conda deactivate`.

Je veux…	Commande
Installer ou mettre à jour les dépendances	`pip install -r requirements-dev.txt`
Rendre le code `velov` importable	`pip install -e .`
Recréer un environnement cassé	`rm -rf .venv` puis `/opt/anaconda3/bin/python -m venv .venv`, puis les deux lignes ci-dessus
Ne plus activer Anaconda automatiquement	`conda config --set auto_activate_base false`

Les données

Je veux…	Commande	Où
Générer l'historique simulé (90 jours, suffit pour le Must du TP4)	`python -m velov.data`	Mac
Collecter une photo des vraies données (relevés, fiches, météo) dans le bronze	`python -m velov.ingest`	Mac ou VM
Transformer le bronze en silver et gold, puis tester	`cd dbt && dbt build`	Mac ou VM
Vérifier la configuration dbt	`cd dbt && dbt debug`	Mac ou VM
Ouvrir la base DuckDB en SQL	`duckdb data/velov.duckdb` (quitter : `.quit`)	Mac ou VM

Deux règles pour éviter les erreurs déjà rencontrées :

- dbt se lance **depuis le dossier** `dbt/` ; `velov.ingest` peut se lancer d'où tu veux, il écrit toujours dans `data/velov.duckdb` à la racine.
- **Quitter le client DuckDB** avant de lancer `velov.ingest` ou `dbt build` : un seul programme à la fois peut écrire dans la base (sinon `Could not set lock on file`).

Requêtes utiles dans le client DuckDB :

```
SELECT count(*) FROM bronze_station_status;
SELECT max(loaded_at) FROM bronze_station_status;   -- dernière collecte
SELECT count(*) FROM station_hourly;                  -- lignes du gold
SELECT min("timestamp"), max("timestamp") FROM station_hourly;  -- période couverte
```

Les données du Mac et celles de la VM sont **deux bases différentes** : `data/` n'est pas dans git. L'historique réel se construit sur la VM, avec la collecte planifiée.

La collecte planifiée sur la VM

Pas encore en place : à faire une fois, pour que l'historique se construise jour et nuit. Toutes les commandes se lancent **sur la VM** (`ssh 192.168.1.232`).

1. Créer l'environnement Python du projet sur la VM :

`bash cd ~/velov-mlops python3 -m venv .venv .venv/bin/pip install -r requirements-dev.txt .venv/bin/pip install -e .`

Si `python3 -m venv` répond que `ensurepip` ou `python3-venv` manque, le paquet n'est pas installé sur la VM et il faut les droits administrateur : demander au prof. 2. Tester une collecte à la main :

`bash mkdir -p logs .venv/bin/python -m velov.ingest` 3. Planifier : `crontab -e`, puis ajouter cette ligne à la fin du fichier :

`*/5 * * * * cd ~/velov-mlops && .venv/bin/python -m velov.ingest >> logs/ingest.log 2>&1`

`*/5 * * * *` = toutes les 5 minutes. `>> logs/ingest.log 2>&1` garde la sortie et les erreurs dans un fichier, puisque personne ne regarde le terminal.

Je veux…	Commande (sur la VM)
Voir les tâches planifiées	`crontab -l`
Suivre la collecte en direct	`tail -f ~/velov-mlops/logs/ingest.log` (quitter : `Ctrl + C`)
Vérifier la dernière collecte	`cd ~/velov-mlops && .venv/bin/python -c "import duckdb; print(duckdb.connect('data/velov.duckdb', read_only=True).sql('SELECT max(loaded_at) FROM bronze_station_status'))"`

À savoir : le `git pull` ne touche pas `data/` ni `logs/` (hors de git), la collecte continue après une mise à jour du code. Ajouter `logs/` au `.gitignore` sur le Mac.

Entraîner, tracer, tester (Mac)

Dire une fois par terminal où est le serveur MLflow, sans jamais l'écrire dans le code :

```
export MLFLOW_TRACKING_URI=http://192.168.1.232:5000
```

Je veux…	Commande
Entraîner sur les données simulées	`python -m velov.train`
Entraîner sur la table gold (vraies données)	`python -m velov.train --data data/velov.duckdb`
Historique encore court	ajouter `--test-days 2`
Numéroter la version du modèle	ajouter `--version 0.2.0`
Tracer l'entraînement dans MLflow	ajouter `--mlflow`

Je veux…	Commande
Lire les métriques du dernier modèle	`python -c "import json; print(json.load(open('models/metadata.json'))['metrics'])"`
Voir les runs	navigateur : `http://192.168.1.232:5000`
Lancer tous les tests	`pytest`
Vérifier le style du code	`ruff check .`

Ce qui arrive avec l'étape 2 du TP4 (`velov/tracking.py`, pas encore écrit) : l'option `--seed` et un run complet (paramètres, métriques de test, commit git, empreinte des données). L'énoncé demande ensuite `python -m velov.train --mlflow --seed 42` deux fois, puis avec `--seed 7`, pour comparer.

Avant un run qui compte : **commiter**. Un run lancé avec du code non commité n'est pas rejouable (il sera marqué `git_dirty`).

Les stacks Docker

MLflow, sur la VM

Le fichier `.env` de la VM contient `MLFLOW_DB_PASSWORD` (déjà fait, jamais dans git). Toutes ces commandes se lancent dans `~/velov-mlops` sur la VM, ou depuis le Mac en les préfixant par `ssh 192.168.1.232 "cd velov-mlops && ..."`.

Je veux…	Sur la VM (dans `~/velov-mlops`, après `ssh 192.168.1.232`)	Depuis le Mac, avec le contexte
Démarrer ou mettre à jour	`git pull origin tp4` puis `docker compose -f compose.mlflow.yaml up -d --build`	`MLFLOW_PORT=5000 docker --context vm compose -f compose.mlflow.yaml up -d --build` (fichiers du Mac, pas de `git pull`)
Voir l'état (les deux doivent être `healthy`)	`docker compose -f compose.mlflow.yaml ps`	`docker --context vm compose -f compose.mlflow.yaml ps`
Lire les messages du serveur	`docker compose -f compose.mlflow.yaml logs mlflow`	`docker --context vm compose -f compose.mlflow.yaml logs mlflow`
Redémarrer (test de persistance)	`docker compose -f compose.mlflow.yaml restart`	`docker --context vm compose -f compose.mlflow.yaml restart`
Arrêter en gardant les données	`docker compose -f compose.mlflow.yaml down`	`docker --context vm compose -f compose.mlflow.yaml down`
Tout effacer, runs et modèles compris (irréversible)	`docker compose -f compose.mlflow.yaml down -v`	`docker --context vm compose -f compose.mlflow.yaml down -v`

Avec le contexte, `--context vm` se place juste après `docker`, avant `compose`. Toute commande `compose`, même `ps` ou `logs`, relit le fichier compose : le `.env` du Mac doit donc contenir `MLFLOW_DB_PASSWORD`. Et `down -v` efface bien les volumes **de la VM**.

Déployer une nouvelle version, en une ligne depuis le Mac :

```
git push vm tp4 && ssh 192.168.1.232 "cd velov-mlops && git pull origin tp4 && docker compose -f comp
ose.mlflow.yaml up -d --build"
```

API, PostgreSQL et pgAdmin, en local sur le Mac

Je veux…	Commande
Construire l'image de l'API (version + commit)	`./scripts/build.sh`
Démarrer la stack	`docker compose up -d`
Voir l'état	`docker compose ps`
Tester l'API	`curl http://127.0.0.1:8000/ready`
Arrêter	`docker compose down`

Les deux stacks sont indépendantes : `compose.yaml` pour l'API, `compose.mlflow.yaml` pour MLflow (projet `velov-mlflow`). Le détail de l'API est dans `README_EXECUTION.md`.

Git, rendu et dépannage

Le cycle de travail habituel (Mac, depuis la racine du projet)

```
git status --short
git add <fichiers>          # ou git add -A pour tout le dépôt, où que tu sois
git commit -m "Message clair (IA déclarée : outil, demande, vérification)"
git push                    # GitHub
git push vm tp4             # VM
```

`git add .` n'ajoute que le **dossier courant** : lancé depuis `dbt/`, il oublie le reste du projet.

Le rendu d'un TP

```
git tag tp4
git push origin refs/tags/tp4
```

`refs/tags/` est obligatoire : la branche et le tag s'appellent tous deux `tp4`, et `git push origin tp4` serait ambigu. Un tag du même nom déjà récupéré depuis le dépôt du prof se supprime d'abord en local : `git tag -d tp4`.

Dépannage

Symptôme	Cause probable	Solution
`Permission denied (publickey)`	SSH n'utilise pas la clé `groupe6`	Vérifier le bloc de `~/.ssh/config` et les droits 600 de la clé
`Connection timed out` vers 192.168.1.232	Pas sur le réseau de l'école, ou passerelle en panne	Changer de Wi-Fi ; en attendant, MLflow en local (`docs/tp4_mlflow.md`)
`required variable MLFLOW_DB_PASSWORD is missing`	Le `.env` lu ne contient pas le mot de passe (avec `--context vm`, c'est celui du Mac)	Ajouter la ligne `MLFLOW_DB_PASSWORD=...`, identique à celle de la VM
`no such service: build`	`build` écrit sans tirets : Docker croit que c'est un nom de service	`--build`, avec deux tirets
`No module named 'velov'` ou `'duckdb'`	Mauvais Python (Anaconda)	`conda deactivate` puis `source .venv/bin/activate`
`Could not set lock on file`	Le client DuckDB a la base ouverte	`.quit` dans le client
`Table ... does not exist` (dbt)	Le bronze n'a jamais été collecté sur cette machine	`python -m velov.ingest`

Symptôme	Cause probable	Solution
`handshake operation timed out`	Serveur Vélo'v ou réseau lent	Relancer ; en planifié, le passage suivant rattrape
`Invalid Host header` (403) sur MLflow	Adresse non autorisée par `--allowed-hosts`	Utiliser l'IP de la VM ou `localhost`
`src refspec tp4 matches more than one`	Branche et tag de même nom	`git push origin refs/tags/tp4`
`Split vide` à l'entraînement	Pas assez d'historique	`--test-days` plus petit, ou laisser la collecte tourner